

METRo Developer README file

You are currently reading the Developer README file. For general information about METRo please read the [README](#) file.

Setting up development environment

`init_devel.sh`

You need to run the `init_devel.sh` script to prepare your working directory. If the script complain about the environment variable `PYTHON_INCLUDE`, you need to set it to a valid value. Usually, it will be something like: `export PYTHON_INCLUDE=/usr/include/pythonX.X` where the X's are the version numbers. You only need to run the `init_devel.sh` script one time.

- From now on, we assume the `init_devel.sh` script has been run.

New optional features

Freezing point of water in the forecast

The atmospheric forecast file can optionally give METRo the freezing point of water (in Celsius) for each forecast timestep, instead of always assuming the physical freezing point of pure water (0 C). This is useful for roads treated with de-icing chemicals, where water on the surface does not actually freeze at 0 C.

1. Add a `<tfz>` element (Celsius) to every `<prediction>` in the forecast file:

```
<prediction>
  <forecast-time>2018-03-27T15:00Z</forecast-time>
  <at>-5.0</at>
  ...
  <tfz>-3.0</tfz>
</prediction>
```

2. Run METRo with the option `--use-freezing-point-forecast`:

```
metro --use-freezing-point-forecast --input-forecast forecast.xml \
      --input-observation observation.xml --input-station station.xml \
      --output-roadcast roadcast.xml
```

If the option is not used, or a given prediction has no `<tfz>`, METRo falls back to the physical freezing point of pure water (0 C), i.e. the previous behaviour is unchanged.

Custom road layer type

The station configuration file's road layers were previously restricted to 4 built-in materials (asphalt, crushed rock, cement, sand). A 5th type, **custom**, lets you supply the thermal capacity (J/m³/K) and conductivity (W/m/K) of the layer directly, for materials METRo doesn't know about.

1. In the station file, set `<type>custom</type>` and add `<capacity>` and `<conductivity>` to that roadlayer:

```
<roadlayer>
  <position>1</position>
  <type>custom</type>
  <thickness>0.15</thickness>
  <capacity>1.8e6</capacity>
  <conductivity>1.2</conductivity>
</roadlayer>
```

2. Run METRo normally; no command line option is needed, the layer type in the station file is enough.

Both `<capacity>` and `<conductivity>` are mandatory for a **custom** layer, and must fall within a validated range:

```
capacity:      1.0e6 to 1.0e7   J/m3/K
conductivity:  0.05  to 3.5     W/m/K
```

Values outside this range are rejected at startup with a clear error message. This is not an arbitrary restriction: METRo's ground grid (`src/model/grille.f`) is a fixed-size, unbounded-checked array tuned for the 4 built-in materials. Values far outside the validated range make it numerically unstable and crash the compiled physics core (segfault) instead of failing gracefully, so keep new materials within (or close to) this range.

Non-**custom** layers ignore `<capacity>/<conductivity>` if present.

Grip / traction index

The roadcast can optionally include a grip index (0.0 = essentially no traction, 1.0 = dry bare pavement) for every prediction, derived from the road condition code (RC), water/ice depth and surface temperature METRo already computes. This is the same approach used by commercial pavement sensors (e.g. Lufft IRS31/NIRS31): grip is never physically measured, it is derived from lookup tables and empirical thresholds over surface state.

1. Run METRo with the option `--output-grip-index`:

```
metro --output-grip-index --input-forecast forecast.xml \
      --input-observation observation.xml --input-station station.xml
\
      --output-roadcast roadcast.xml
```

2. The roadcast gains a `<grip>` element per prediction:

```
<prediction>
  <roadcast-time>2018-03-27T15:20Z</roadcast-time>
  ...
  <rc>7</rc>
  <grip>0.28</grip>
</prediction>
```

If the option is not used, roadcast.xml is unchanged (no `<grip>` element), i.e. the previous behaviour is unchanged.

If `--use-freezing-point-forecast` (see above) is also used, grip already accounts for it: a salted/treated road (lower effective freezing point) keeps a higher grip value for longer below 0 C, since grip is computed from the same RC/temperature values that already reflect the depressed freezing point.

STATUS: the grip formula and its constants (`src/frontend/toolbox/metro_grip.py`) are a first pass, not yet validated against real observations. They encode a plausible qualitative shape (dry > wet > melting snow > mix > ice/frost > icing rain; ice friction worst near the freezing point, recovering somewhat when colder) but the actual numbers were not calibrated against any sensor or field data. Treat grip output as provisional until validated.

JSON roadcast output

The roadcast can optionally also be written as JSON, alongside the existing XML file, for anyone consuming METRo output today.

```
metro --output-roadcast-json roadcast.json --input-forecast forecast.xml \
  --input-observation observation.xml --input-station station.xml \
  --output-roadcast roadcast.xml
```

Produces a file shaped like:

```
{
  "header": {"version": "1.6", "road-station": "rsy", ...},
  "prediction-list": [
    {"roadcast-time": "2018-03-27T15:20Z", "rc": 1, "st": -0.83, ...},
    ...
  ]
}
```

Field names match the XML tags exactly. If `--output-roadcast-json` is not given, no JSON file is written and XML output is unchanged.

Limitation: multi-value fields (currently only TL / `--output-subsurface-levels`) are not supported in JSON output and are skipped with a warning if present.

Refined precipitation type (freezing rain/drizzle)

METRo's atmospheric forecast internally classifies each timestep's precipitation as rain (1) or snow (2), based on which of `<ra>/<sn>` is increasing (falling back to air temperature when neither is). This has no freezing rain/drizzle distinction: liquid precipitation falling at a subfreezing air temperature was simply "rain". This does not change how the road model computes the icing-rain road condition (that already comes from the road surface's own temperature relative to the freezing point, independently of the precipitation label) - it only adds visibility into *why* the road turned icy.

Enable it to add a diagnostic `<precip-type>` field (1=rain, 2=snow, 3=freezing rain/drizzle) to the roadcast:

```
metro --output-precip-type --input-forecast forecast.xml \
      --input-observation observation.xml --input-station station.xml \
      --output-roadcast roadcast.xml
```

If not used, roadcast.xml is unchanged.

Config file as JSON

`--config` and `--generate-config` now also accept a JSON file, detected by the `.json` extension - no new command line option needed:

```
metro --generate-config myconfig.json ...
metro --config myconfig.json --input-forecast forecast.xml ...
```

A JSON config file is a flat `{"KEY": value}` object (see `myconfig.json` after running `--generate-config` for the full list of keys). Anything with a non-`.json` extension is still read/written as the original Plist-XML format. Note JSON has no comment syntax, so unlike the Plist-XML writer, the generated JSON file does not include the per-key COMMENTS documentation.

Unit tests

`tests/` has `stdlib-unittest` coverage (no new dependency - the CI environment does not install `pytest`) for a few individual modules previously only exercised end-to-end through `test_suite.py`'s XML-diff regression cases: config merging (`metro_config.overlay_config`), the `Metro_data` matrix operations (append/get/set/del column and row), a handful of `metro_util` helpers, and the grip formula.

Run with:

```
python3 -m unittest discover -s tests -t .
```

`tests/__init__.py` sets up `sys.path` and import order deliberately: `metro_util.py` and `metro_error.py` import each other, and importing `metro_util.py` directly as the very first module in a process crashes (`AttributeError`, circular import) - see the comment in `tests/__init__.py` for why, and don't change that import order without understanding it first.

Compiling "model"

Fortran model

To compile the Fortran model you will need to perform the following action:

1. `cd src/model/`
2. `.../.../scripts/do_macadam`

Building package

1. Change the METRo version number in the following files:

- `scripts/do_macadam`
- `src/frontend/metro_config.py`

2. Change release date in the following files:

- `README`
- `src/frontend/metro_config.py`

3. Commit every change

```
svn commit
```

4. Compile model:

compile the Fortran model (see [Compiling "model"](#))

5. Update package list

```
grep -E "^src/frontend/.*\.py" scripts/make_package.py | sort > /tmp/packaged_modules
find src/frontend -name *.py | sort > /tmp/actual_modules
diff -y -W 200 /tmp/packaged_modules /tmp/actual_modules
vi scripts/make_package.py
```

6. Build Package

```
cd scripts/
./make_package.py
```

- You need a valid gpg key to be able to sign your package

Release

1. Make a tag in SVN for the release of that version. You will need an access to the metro project on GNA to do that. The following is an example of the user Francois Fortin making a tags for the 3.2.4 version

```
svn copy svn+ssh://francois_fortin@svn.gna.org/svn/metro/metro/trunk \  
svn+ssh://francois_fortin@svn.gna.org/svn/metro/metro/tags/metro-3.2.4
```

2. Upload package to the GNA web site (<http://download.gna.org/metro/>)

```
scp metro-3.2.4.tar.bz2 metro-3.2.4.tar.bz2.sig \  
francois_fortin@download.gna.org:/upload/metro/
```

3. Create a new LATEST_RELEASE file with the new version number. The content of the file is:

Please note:

- The latest stable version of the METRo package is:
metro-3.2.4

4. Upload the LATEST_RELEASE file to the GNA web site (<http://download.gna.org/metro/>)

```
scp LATEST_RELEASE francois_fortin@download.gna.org:/upload/metro/
```